Data Science and Artificial Intelligence

Machine Learning

Multi layer Perceptron

Lecture No. 3

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic — MLP
- Topic — SLP
- Topic
- Topic
- Topic

Topics to be Covered

- Topic — MLP- FFNN
- Topic — Backpropagation
- Topic
- Topic
- Topic

REVISION

MLP Hidden layer ⇒ input layer hidden layer & out layer.

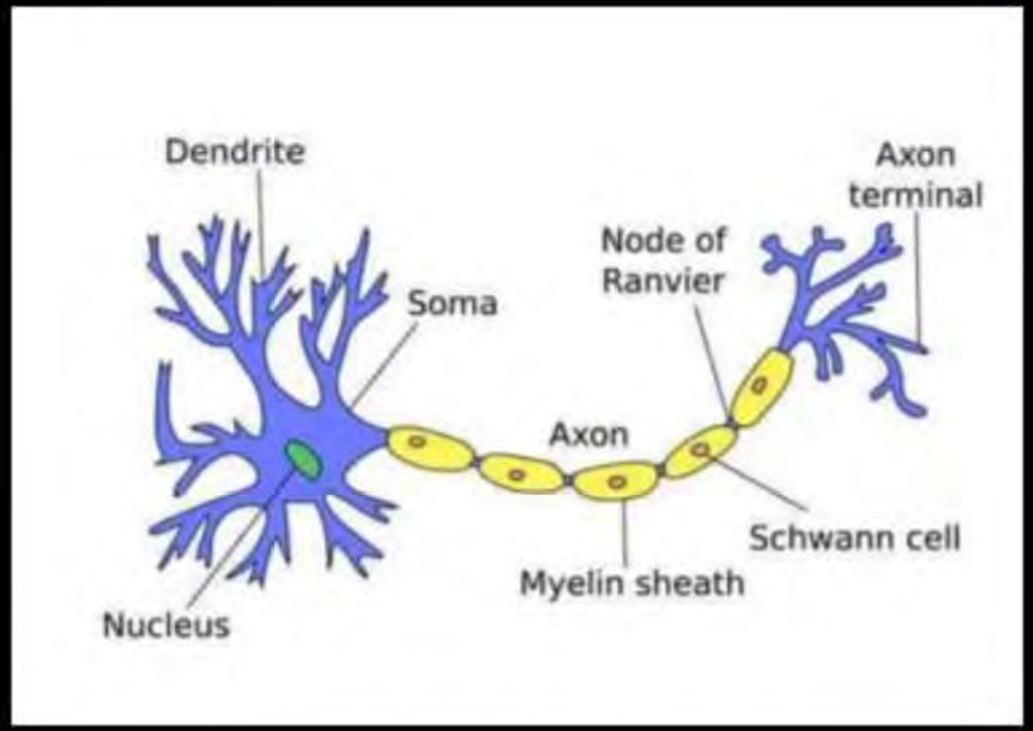
SLP input & out layer ⇒ linear classification

Perceptron learning

Neural Networks

How weights are assigned in the neurons …

- work similar to neural n/w of human body.



Single Vs Multi Layer Perceptrons

Single Layer Perceptron has just two layers of input and output. It only has single layer hence the name single layer perceptron.

Perceptron learning

Single Vs Multi Layer Perceptrons

- Limitations of the Perceptron

- Linear Separability: The perceptron can only solve problems where the data is linearly separable. If the data cannot be separated by a straight line (or hyperplane in higher dimensions), the perceptron will not converge.
- Non-linear Problems: For problems like the XOR problem, a single-layer perceptron fails as it cannot capture the non-linearity.

Perceptron learning

Single Vs Multi Layer Perceptrons

- A multilayer perceptron is a type of feed-forward artificial neural network that generates a set of outputs from a set of inputs.
- An MLP is a neural network connecting multiple layers in a directed graph, which means that the signal path through the nodes only goes one way. The MLP network consists of input, output, and hidden layers.
- Each hidden layer consists of numerous perceptron's which are called hidden layers or hidden unit.

Single Vs Multi Layer Perceptrons

Multilayer Perceptron (MLP):

- Consists of multiple layers of neurons.
- Can solve non-linear problems using hidden layers and non-linear activation functions.
- Trained using backpropagation.

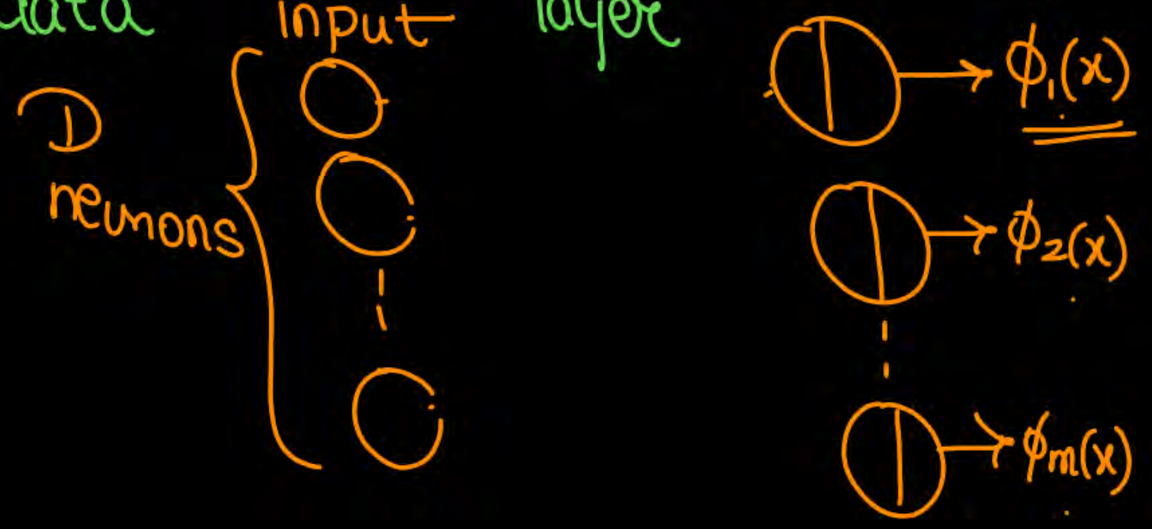
Perceptron learning

Multi Layer Perceptrons

What the hidden layer does ??

→ hidden layer help in mapping, Change dimensionality

D dimensional data → Hidden layer M neurons,



Multi Layer Perceptrons

How can we generate a transformation function from R (D) to R(M)

So From

$$(x_i^1, x_i^2 ---- x_i^D) \longrightarrow \underbrace{(\phi_1(x_i), \phi_2(x_i) --- \phi_m(x_i))}$$

2D point $\Rightarrow (- \; -) \Rightarrow R^2$

D dimensional Point

$(- - - - -) \Rightarrow R^D$

$\Rightarrow$ all activation fxn can be same but since weights are different for each neurons thus these values will differ

Perceptron learning

Multi Layer Perceptrons

So the linear classifier in the new dimension space seems to be non linear in lower/original dimension space

- after dimensionality change ⇒
  - in R^M space if we have linear classifier, then that classifier in original space is N.L
- MLP is a type of NN,

Perceptron learning

Feed Forward NN

Signal flow is unidirectional, from I/P to O/P.

- A Feedforward Neural Network (FNN) is a type of artificial neural network where the connections between nodes do not form a cycle.
- It is the simplest form of artificial neural network, consisting of an input layer, one or more hidden layers, and an output layer.
- Information moves in one direction—from the input layer, through the hidden layers, to the output layer.

• there is no feedback, signal flows only in one direction.

Perceptron learning

Feed Forward NN

- Components:
- Input Layer: Consists of input neurons that receive the initial data. → no processing only forward data
- Hidden Layers: Consist of neurons that process inputs from the previous layer using weights, biases, and activation functions. Can be multiple hidden layers, each adding more complexity and capacity to the network.
- Output Layer: Consists of output neurons that produce the final prediction or classification. → Regression/Classification

Perceptron learning

Feed Forward NN

- Connections:

- Each neuron in one layer is connected to every neuron in the next layer.
- These connections have associated weights that adjust during training.

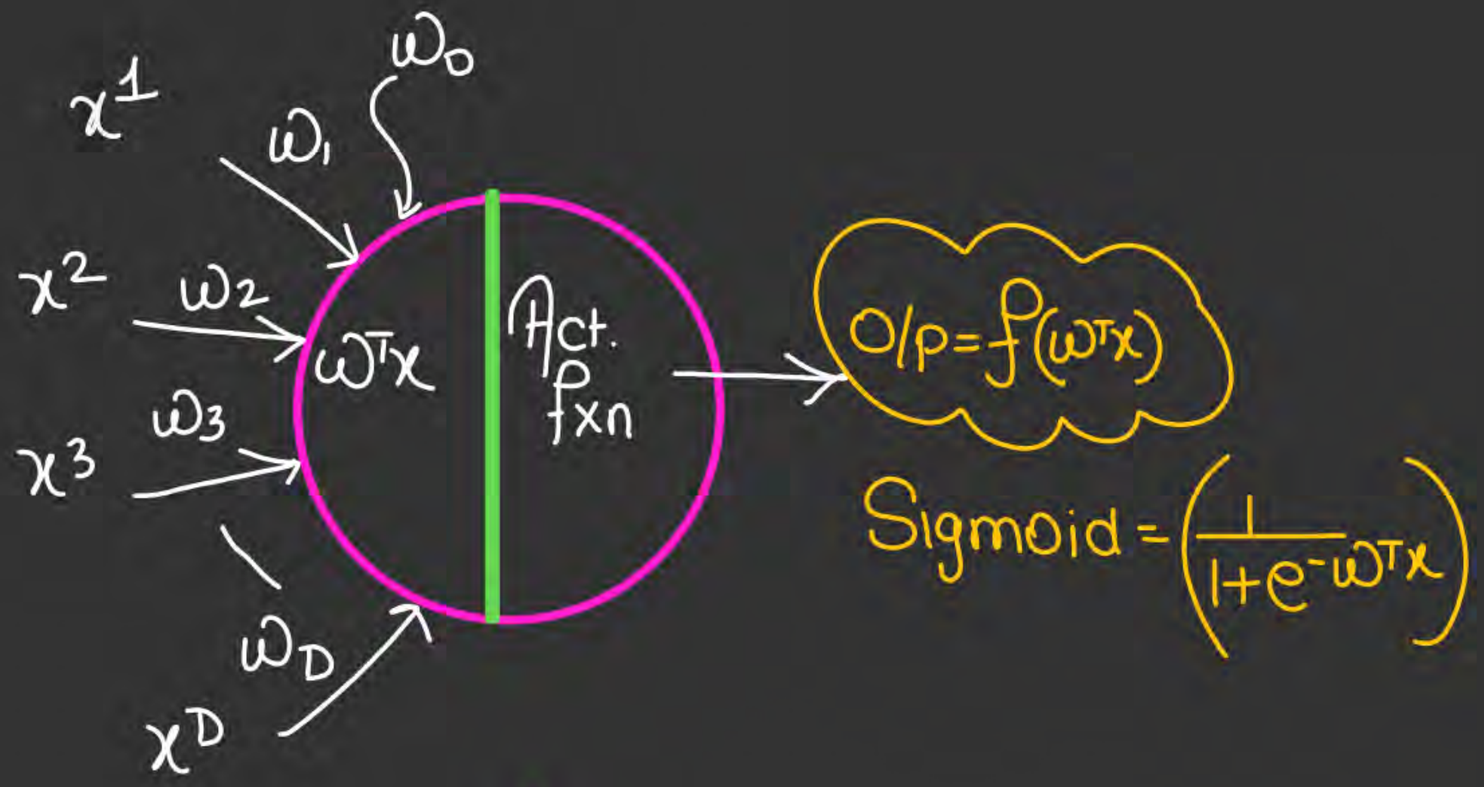
• Why activation fxn has to be NL ⇒

⇒ Becoz this gives Power to Convert NL Seperable data → Linearly Seperable

Perceptron learning

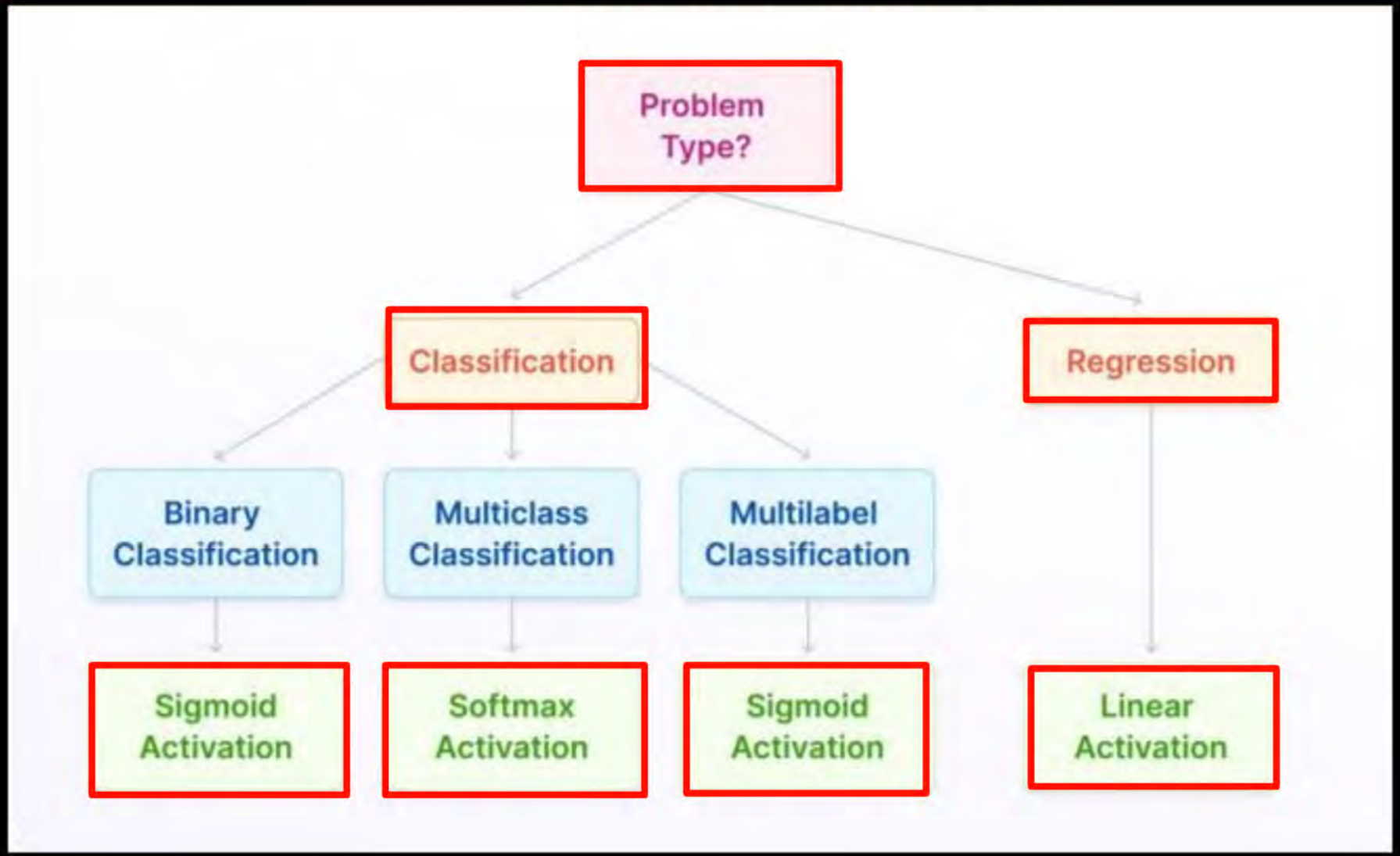
Feed Forward NN

- Activation Functions:
- Activation functions are a critical component of neural networks that introduce non-linearity into the model, allowing networks to learn complex patterns and relationships in the data.
- Activation functions are necessary for neural networks because, without them, the output of the model would simply be a linear function of the input. In other words, it wouldn't be able to handle large volumes of complex data.
- Non-linear functions applied to the weighted sum of inputs.
- Common activation functions include ReLU, sigmoid, and tanh.


x^1
x^2
x^3
x^D
w_0
w_1
w_2
w_3
w_D
$w^T x$
Act. fxn
O/p = f(w^T x)
Sigmoid = (1 / (1 + e^{-w^T x}))

Perceptron learning

Feed Forward NN



Perceptron learning

Multiclass Vs MultiLable ?

Question-1: Multi-class classification problems have multiple categories but each instance is assigned only once.

- True
- False

Comedy
Action
Romance
Sad
Bekar

multilable

Des. me 4
Only one Class
Comedy, Romance, Action

UA
A+
U
A

multiple Classes

Question-2: Multi-label classification problems have each instance can be assigned with multiple categories or a set of target labels.

- True
- False

Perceptron learning

Multiclass Vs MultiLable ?

What is soft max

multi class

- Soft max function ⇒ we have 5 classes

→ Class1

→ Class2

→ Class3

→ Class4

→ Class5

Sigmoid ⇒ $\frac{1}{1+e^{-w^T x}}$

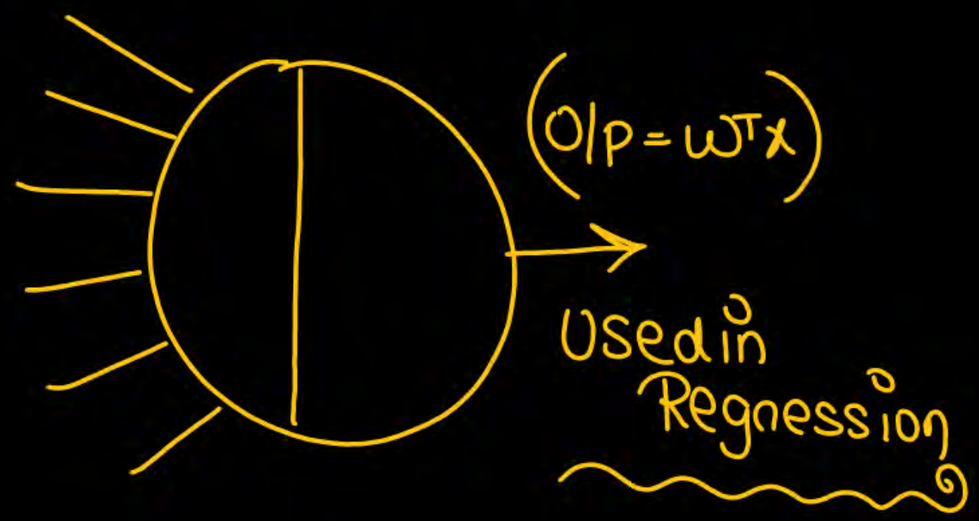
Class1 ← $w^T x > 0$ (Sigmoid > .5)

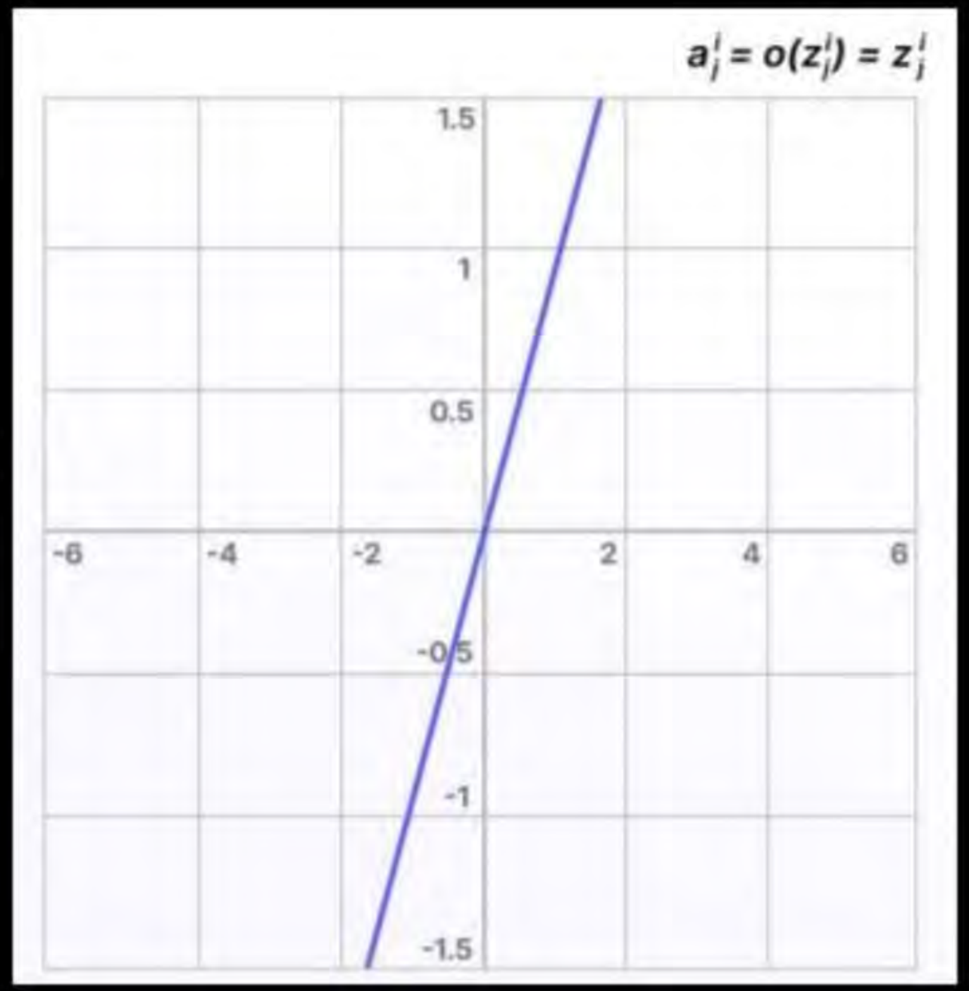
Class0 ← $w^T x < 0$ (Sigmoid < .5)

Binary Class ⇒

Perceptron learning

Linear Activation Functions





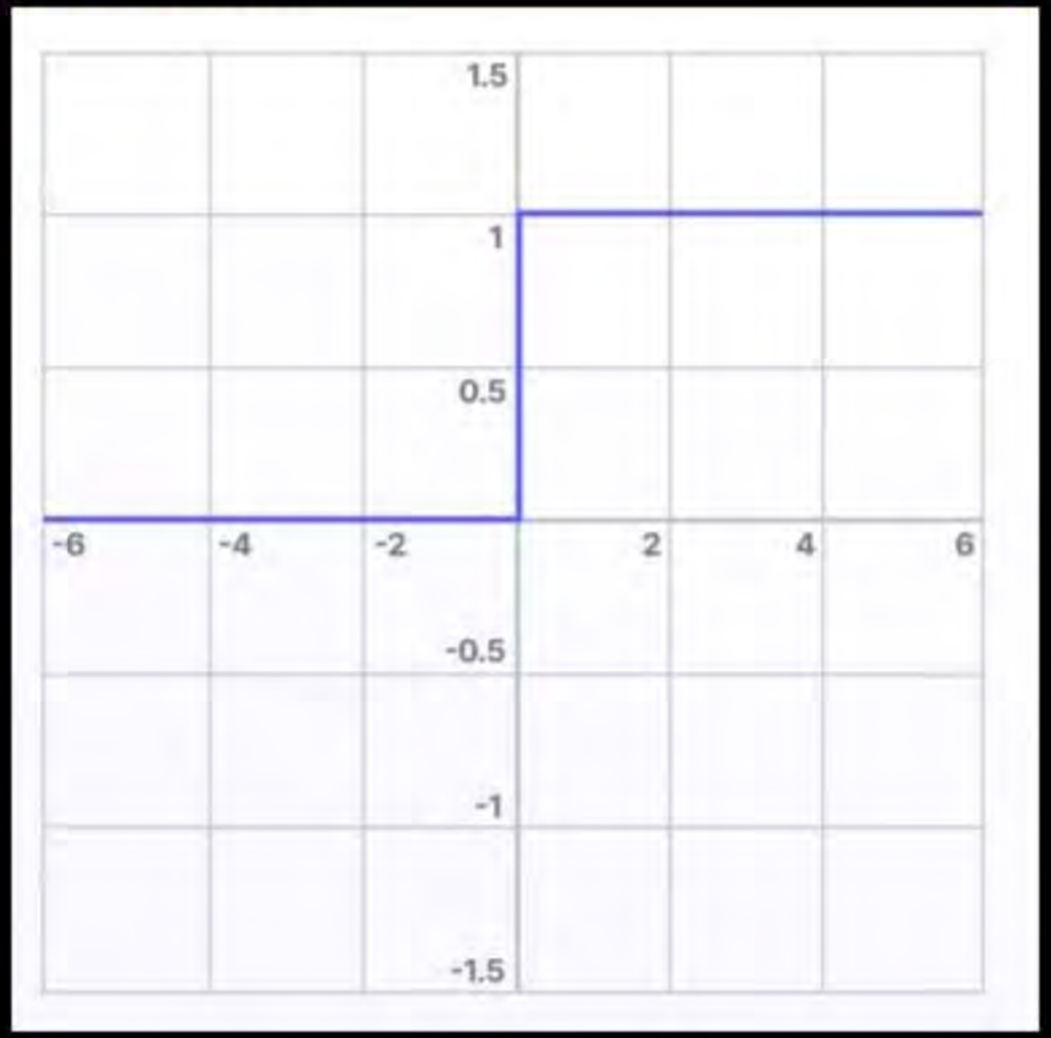
So in Regression Cases hidden layer is still having NL activation fxn.

But o/p layer will use linear activation fxn.

Perceptron learning

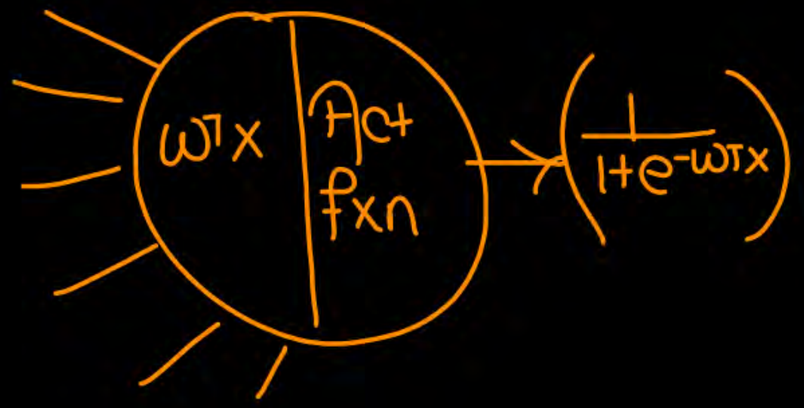
Binary Step Function

- $y = 1, \; w^T x > 0$
  $0, \; w^T x < 0$

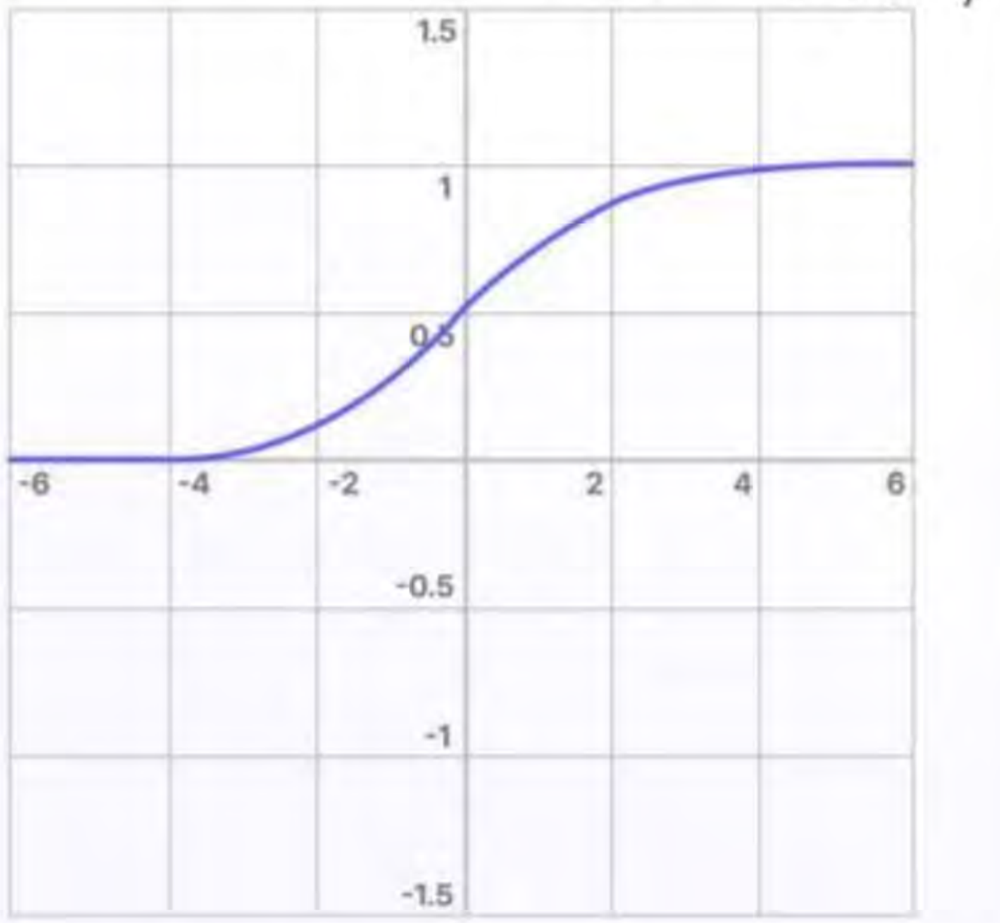


Perceptron learning

Sigmoid, Logistic Activation Functions

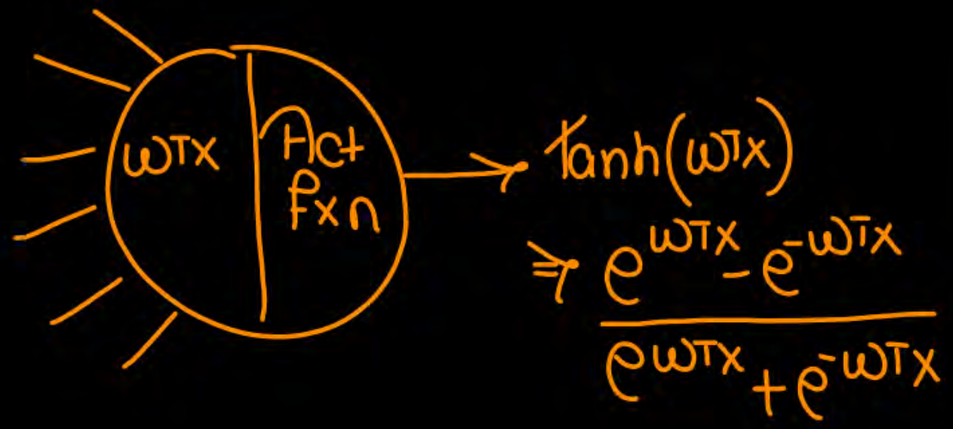


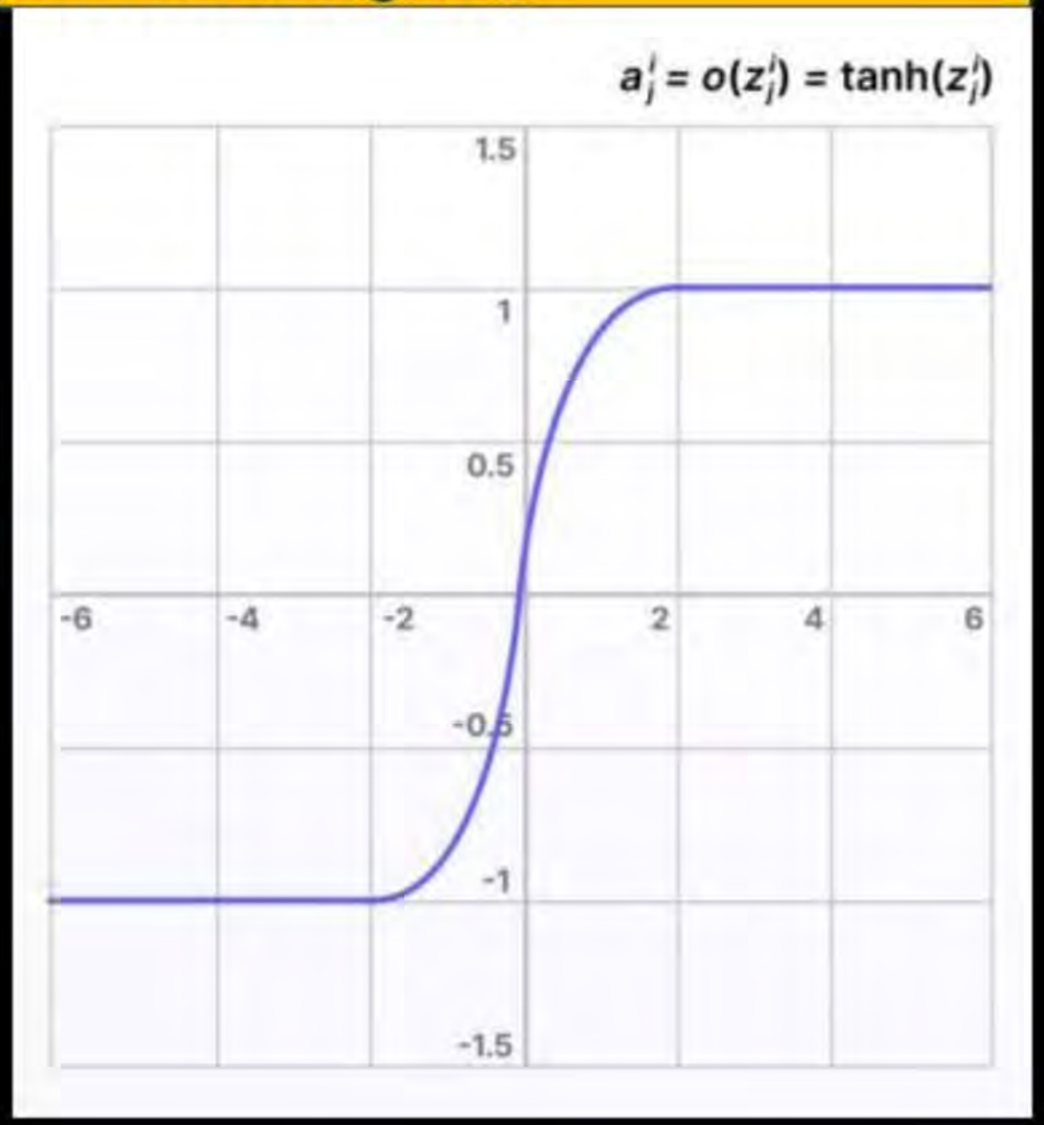
$$a_j^l = o(z_j^l) = \frac{1}{1 + \exp(-z_j^l)}$$



Perceptron learning

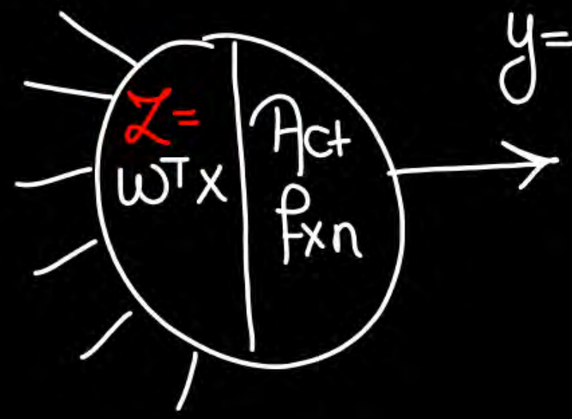
Tanh Function (Hyperbolic Tangent)





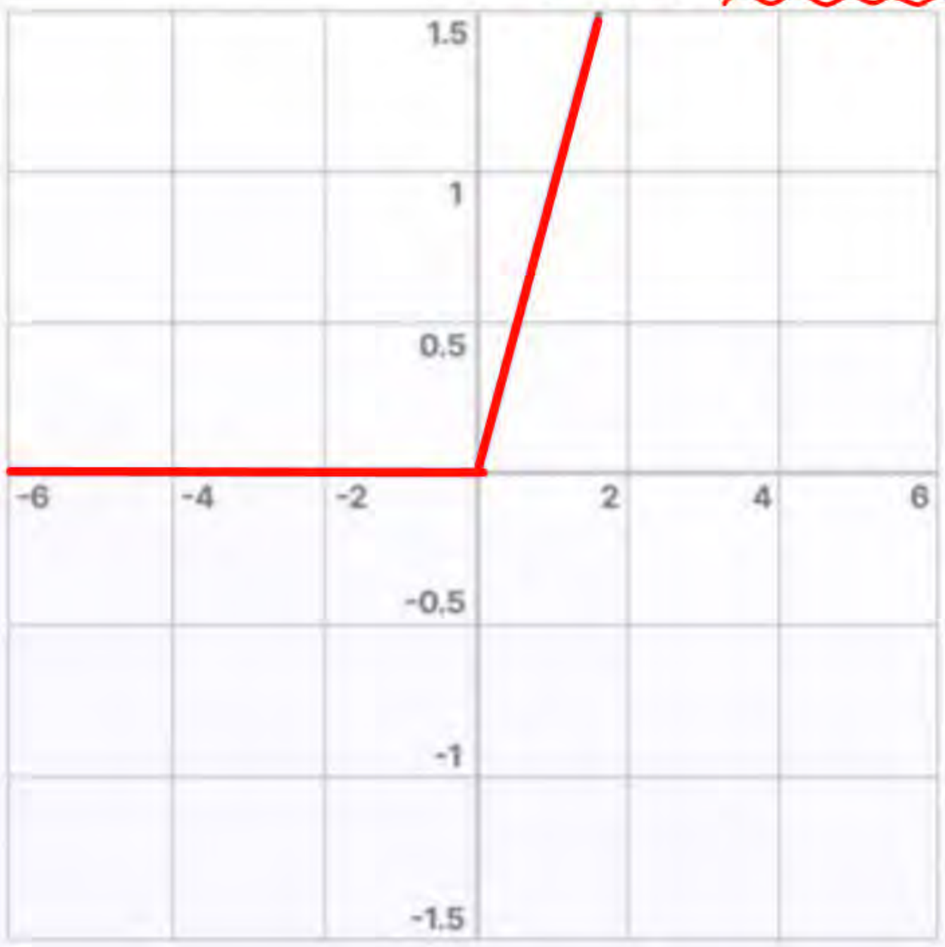
Perceptron learning

Rectified Linear Unit (ReLU)



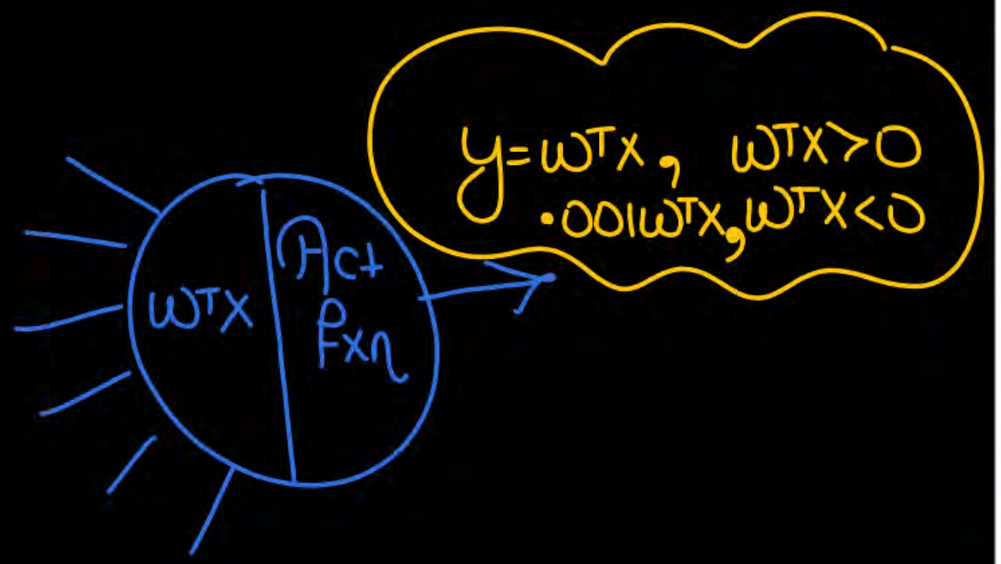
$$y = \begin{cases} w^T x, & w^T x > 0 \\ 0, & w^T x < 0 \end{cases}$$

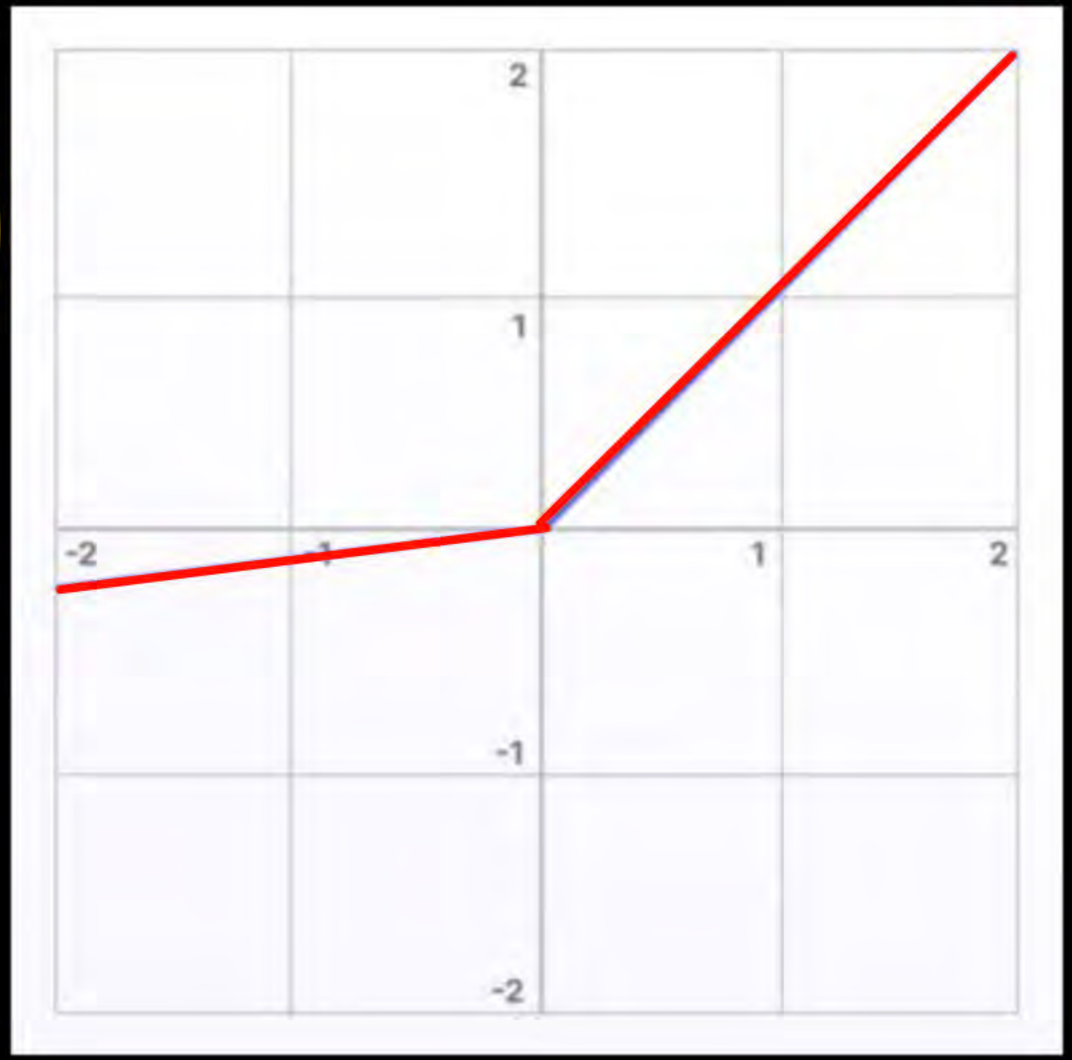
$$a_j^i = o(z_j^i) = \max(0, z_j^i)$$



Perceptron learning

Leaky ReLU Function





How Feedforward Neural Networks Work

- The working of a feedforward neural network involves two phases: the feedforward phase and the backpropagation phase.
- Feedforward Phase: In this phase, the input data is fed into the network, and it propagates forward through the network. At each hidden layer, the weighted sum of the inputs is calculated and passed through an activation function, which introduces non-linearity into the model. This process continues until the output layer is reached, and a prediction is made.
- Backpropagation Phase: Once a prediction is made, the error (difference between the predicted output and the actual output) is calculated. This error is then propagated back through the network, and the weights are adjusted to minimize this error. The process of adjusting weights is typically done using a gradient descent optimization algorithm.

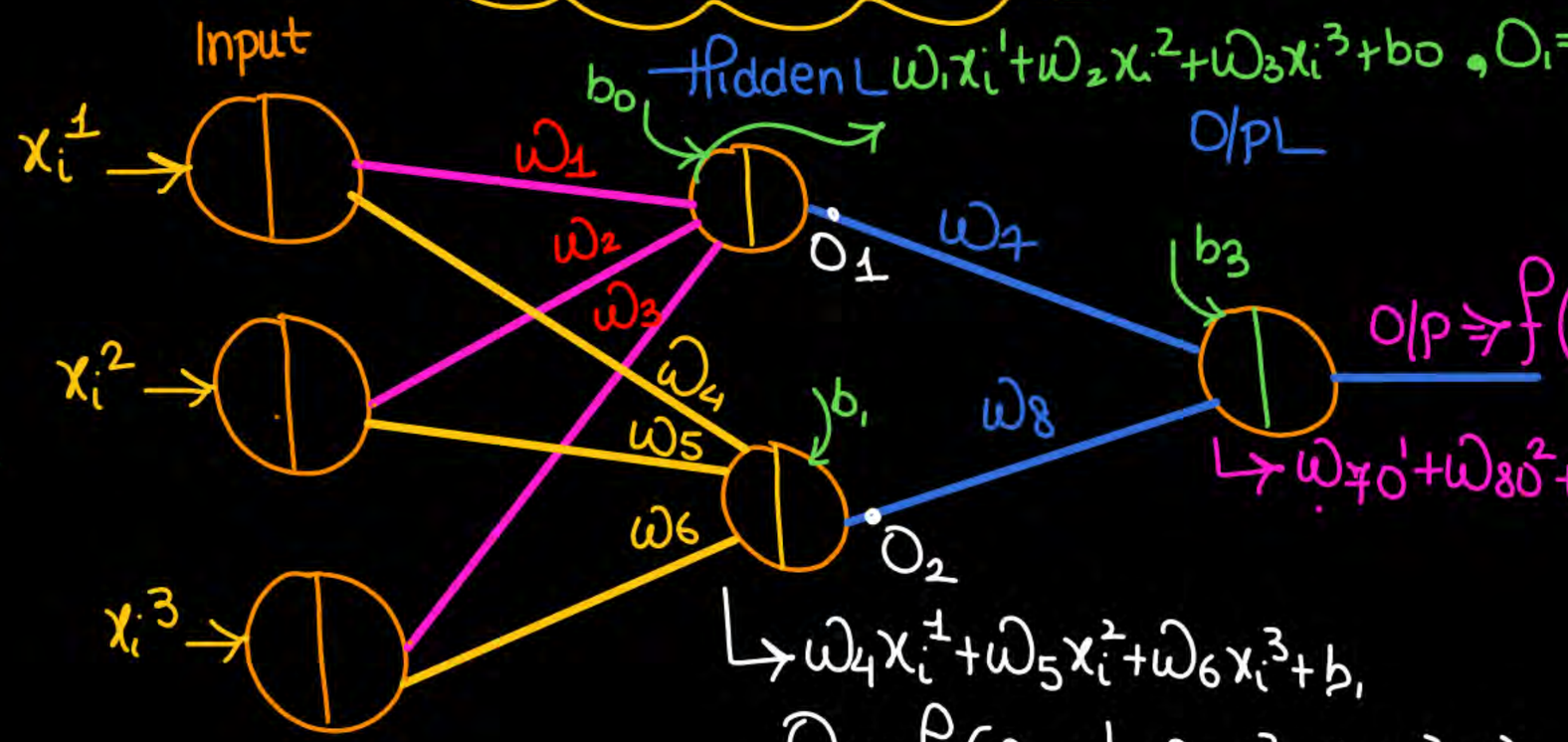
Perceptron learning

• we start with Random weight values.

Feedforward Phase

Ex ⊙

So input is 3D input



$w_1x_i^1+w_2x_i^2+w_3x_i^3+b_0$, $O_1=f(w_1x_i^1+w_2x_i^2+w_3x_i^3+b_0)$

O/P $\Rightarrow f(w_7O^1+w_8O^2+b_3)$

$\hookrightarrow w_7O^1+w_8O^2+b_3$

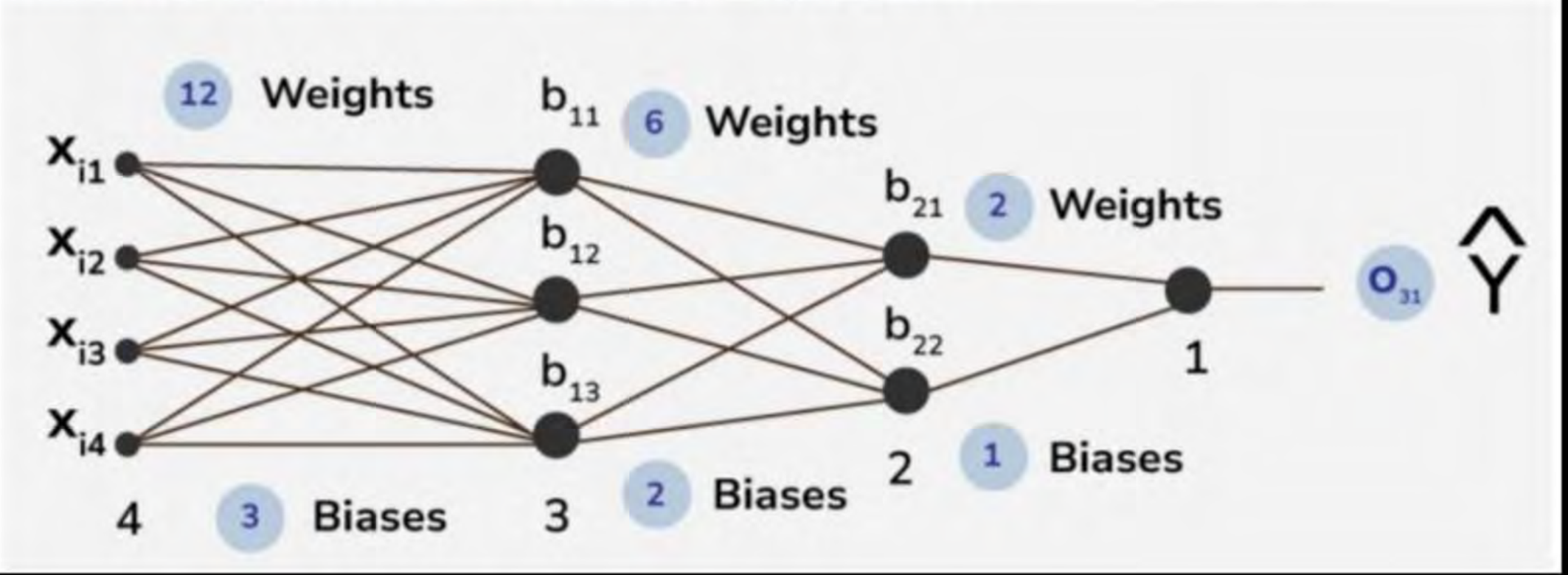
$\hookrightarrow w_4x_i^1+w_5x_i^2+w_6x_i^3+b_1$

$O_2=f(w_4x_i^1+w_5x_i^2+w_6x_i^3+b_1)$

Perceptron learning

Feed Forward NN

Mathematical Explanation of Forward Propagation

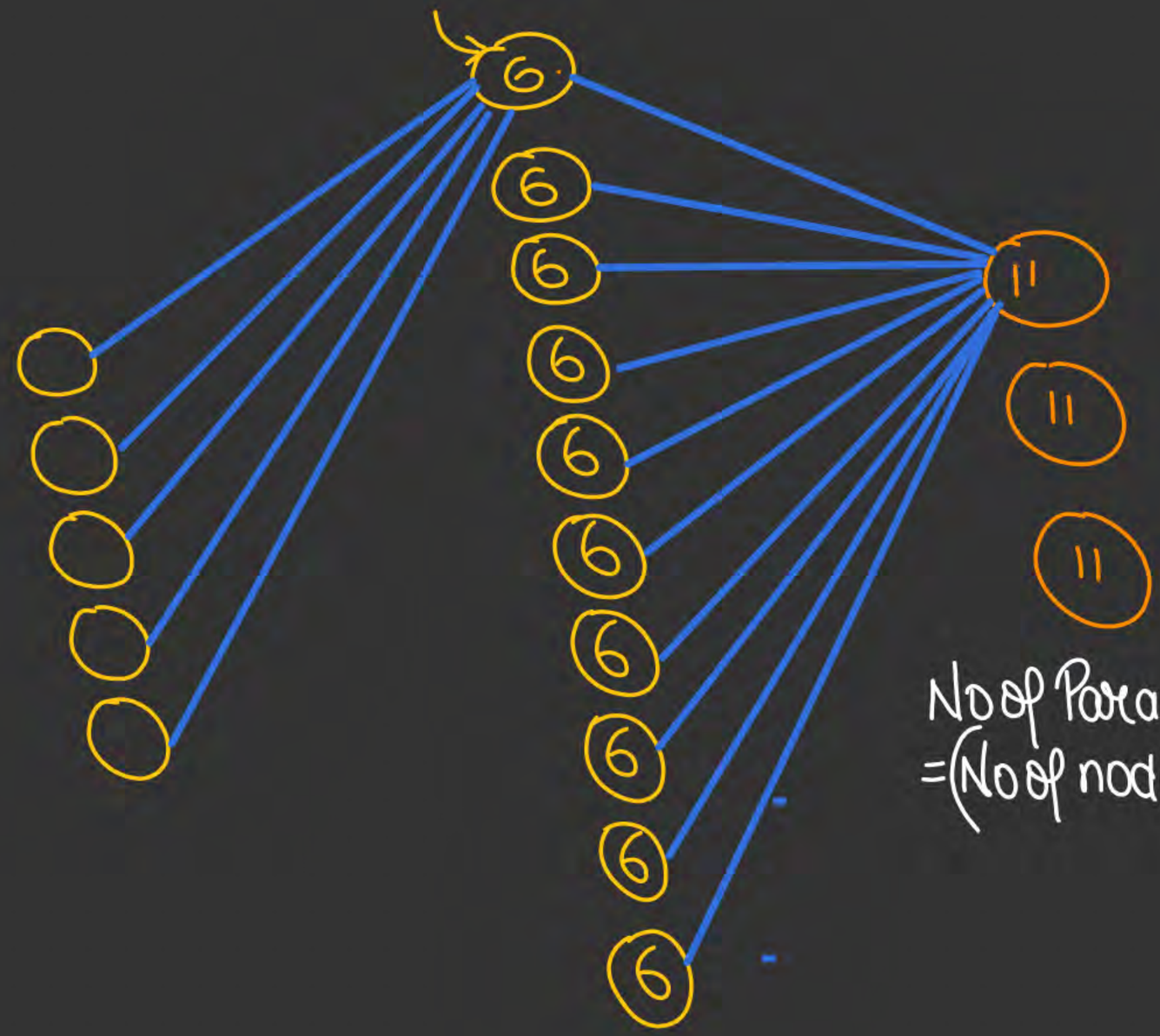


Perceptron learning

Feed Forward NN

Consider a Multi-Layer Perceptron (MLP) model with one hidden layer and one output layer. The hidden layer has 10 neurons, and the output layer has 3 neurons. The input to the MLP is a 5-dimensional vector. Each neuron is connected to every neuron in the previous layer, and a bias term is included for each neuron. The activation function used is the sigmoid function.

Calculate the total number of trainable parameters in this MLP model.



(60+33)

$\Rightarrow$ 93

No of Parameters at any node
=(No of node in previous stage+1)

Q.13	A decision tree classifier learned from a fixed training set achieves 100% accuracy. Which of the following models trained using the same training set will also achieve 100% accuracy? i) Logistic regressor. → RBF Kernel (γ) ii) A polynomial of degree one kernel SVM. iii) A linear discriminant function. iv) Naïve Bayes classifier.
(A)	i
(B)	i and ii
(C)	all of the above
(D)	none of the above

Q.30	For perfectly spherical 2D data centered at the origin, which of the following the pairs of vectors are possible pairs of principal components?
	i) (1, 0) and (0, 1)
	ii) (0, -1) and (-1, 0)
	iii) (1, 1) and (1, -1)
	iv) (-1, 1) and (-1, -1)
(A)	i
(B)	i and iii
(C)	i, ii, and iii
(D)	i, ii, iii and iv

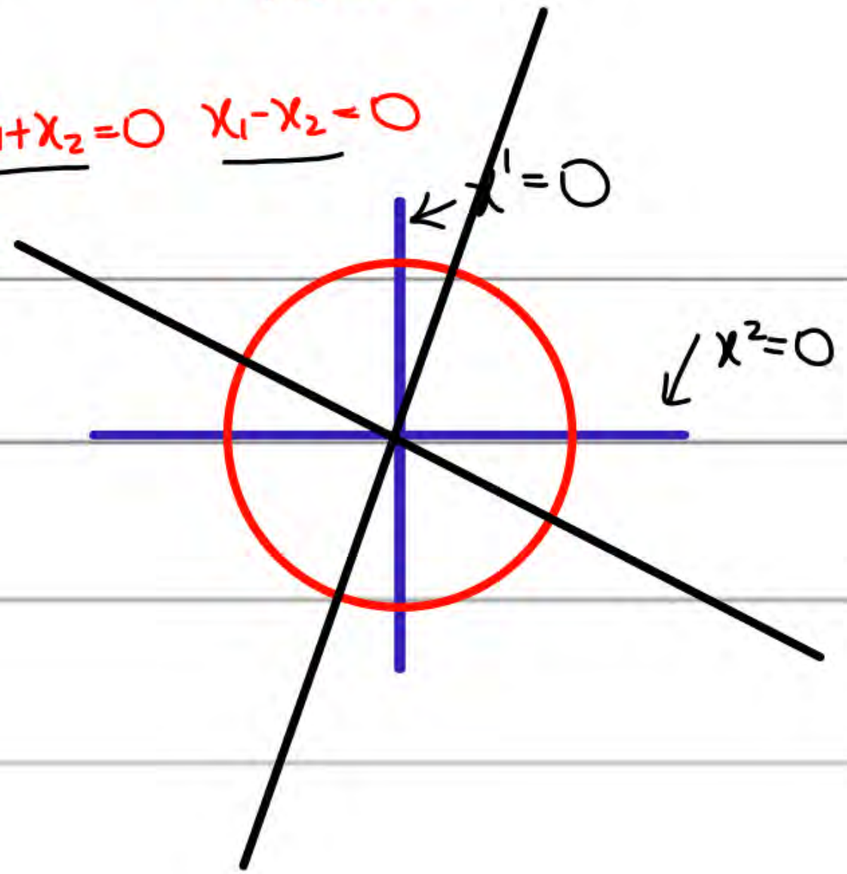
$x_1+0=0$ $\quad$ $0+x_2=0$

$0-x_2=0$ $\quad$ $-x_1+0=0$

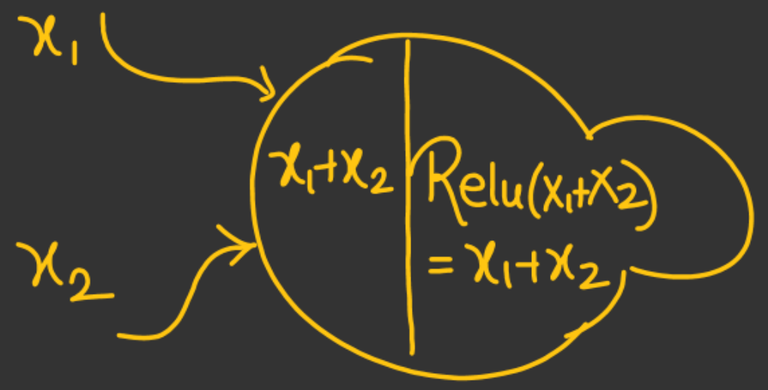
• Visualise

$x_1+x_2=0$ $\quad$ $x_1-x_2=0$

$-x_1+x_2=0$, $-x_1-x_2=0$



$$\mathrm{Relu}(z) = \max(0, z)$$



Consider the two neural networks (NNs) shown in Figures 1 and 2, with *ReLU* activation ($ReLU(z) = \max\{0, z\}, \forall z \in \mathbb{R}$). $\mathbb{R}$ denotes the set of real numbers. The connections and their corresponding weights are shown in the Figures. The biases at every neuron are set to 0. For what values of p, q, r in Figure 2 are the two NNs equivalent, when x_1, x_2, x_3 are positive?

Figure 1

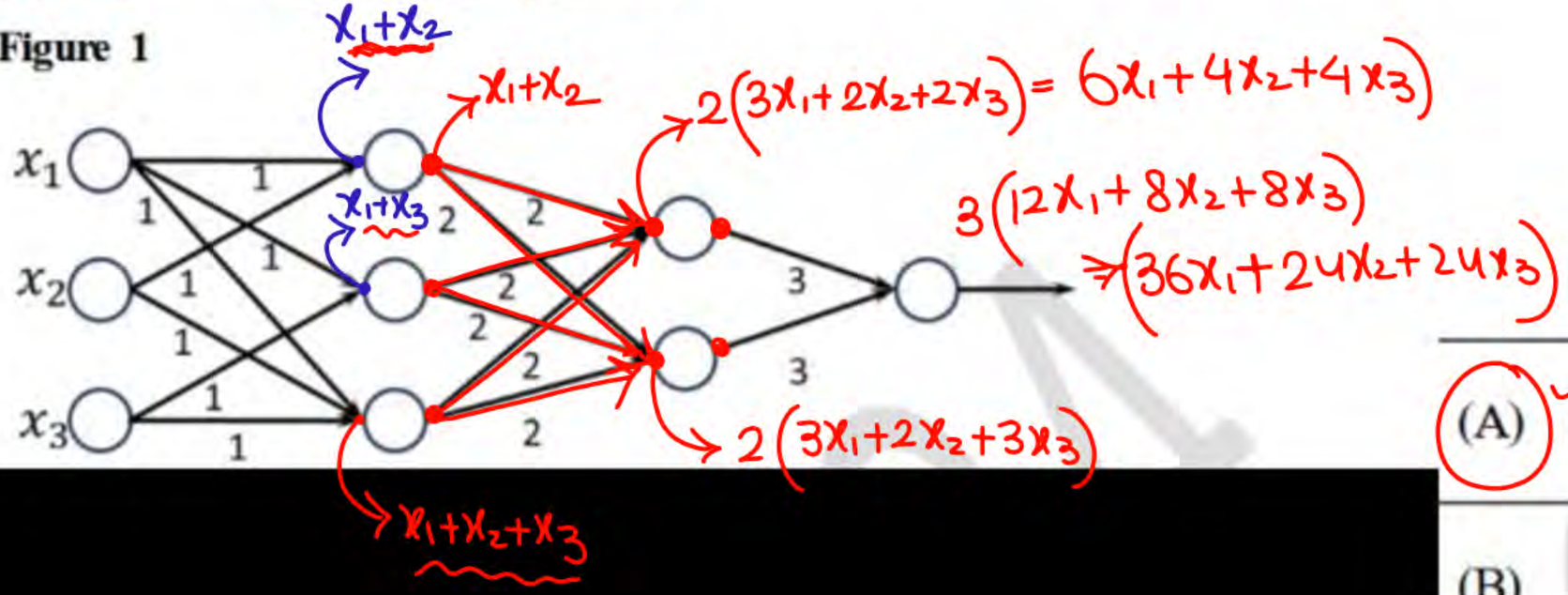
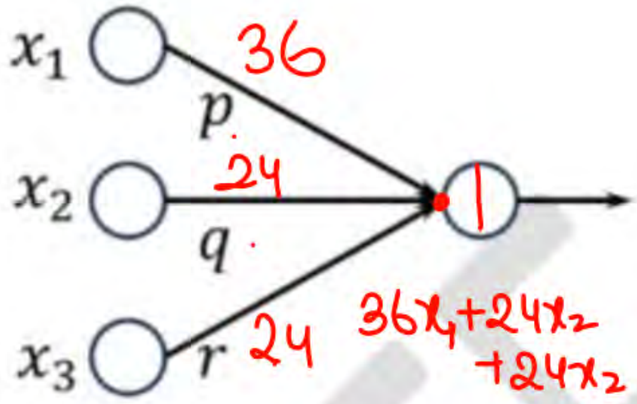


Figure 2



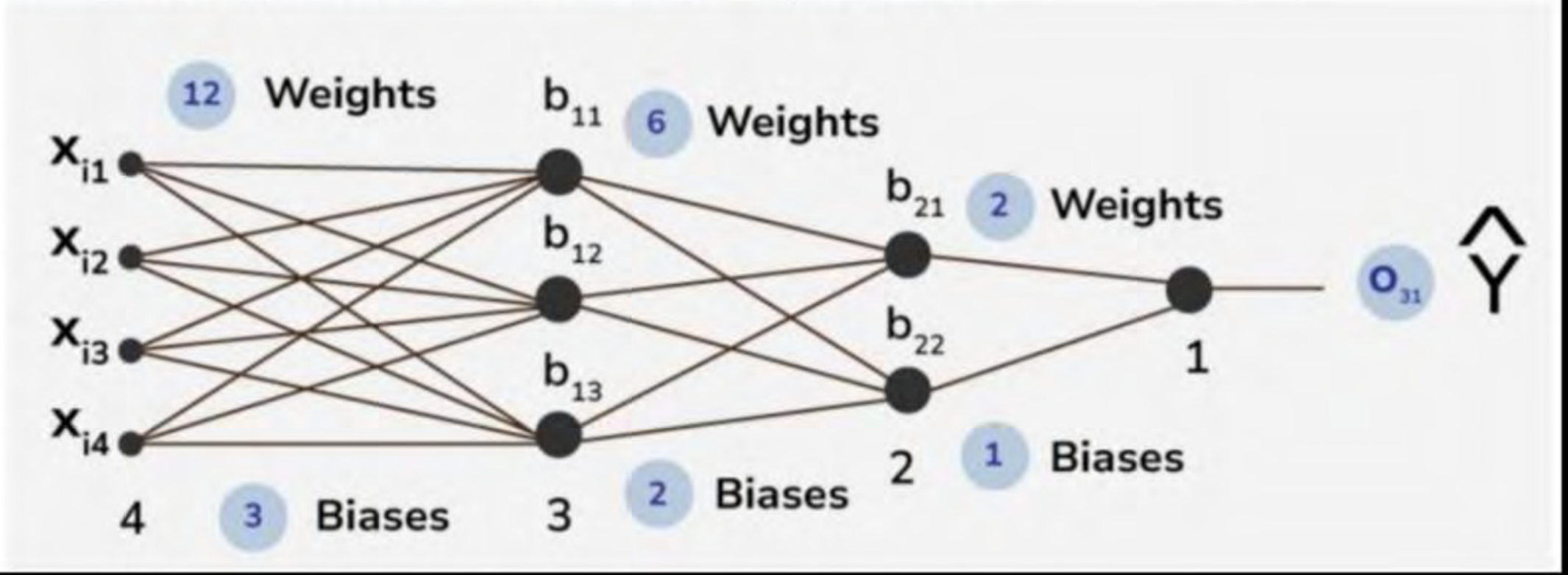
(A)	$p = 36, q = 24, r = 24$
(B)	$p = 24, q = 24, r = 36$
(C)	$p = 18, q = 36, r = 24$
(D)	$p = 36, q = 36, r = 36$

Perceptron learning

SKIP

Feed Forward NN

Mathematical Explanation of Forward Propagation



Perceptron learning

Skip

Inside layer 1

The procedure of the matrix operations for layer 1 are as follows:

$$\sigma\left(\begin{bmatrix} W_{111} & W_{112} & W_{113} \\ W_{121} & W_{122} & W_{123} \\ W_{131} & W_{132} & W_{133} \\ W_{141} & W_{142} & W_{143} \end{bmatrix}^T \begin{bmatrix} X_{i1} \\ X_{i2} \\ X_{i3} \\ X_{i4} \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}\right) = \sigma\left(\begin{bmatrix} W_{111}X_{i1} + W_{121}X_{i2} + W_{131}X_{i3} + W_{141}X_{i4} \\ W_{112}X_{i1} + W_{122}X_{i2} + W_{132}X_{i3} + W_{142}X_{i4} \\ W_{113}X_{i1} + W_{123}X_{i2} + W_{133}X_{i3} + W_{143}X_{i4} \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}\right) =$$

$$\sigma\left(\begin{bmatrix} W_{111}X_{i1} + W_{121}X_{i2} + W_{131}X_{i3} + W_{141}X_{i4} + b_{11} \\ W_{112}X_{i1} + W_{122}X_{i2} + W_{132}X_{i3} + W_{142}X_{i4} + b_{12} \\ W_{113}X_{i1} + W_{123}X_{i2} + W_{133}X_{i3} + W_{143}X_{i4} + b_{13} \end{bmatrix}\right) = \begin{bmatrix} O_{11} \\ O_{12} \\ O_{13} \end{bmatrix} = a^{[1]}$$

Inside layer 2

Inside Layer 2

$$
\begin{aligned}
&\sigma\left(\begin{bmatrix} W_{211} & W_{212} \\ W_{221} & W_{222} \\ W_{231} & W_{232} \end{bmatrix}^T \cdot \begin{bmatrix} O_{11} \\ O_{12} \\ O_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix}\right) \\
&= \sigma\left(\begin{bmatrix} W_{211} \cdot O_{11} + W_{221} \cdot O_{12} + W_{231} \cdot O_{13} \\ W_{212} \cdot O_{11} + W_{222} \cdot O_{12} + W_{232} \cdot O_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix}\right) \\
&= \sigma\left(\begin{bmatrix} W_{211} \cdot O_{11} + W_{221} \cdot O_{12} + W_{231} \cdot O_{13} + b_{21} \\ W_{212} \cdot O_{11} + W_{222} \cdot O_{12} + W_{232} \cdot O_{13} + b_{22} \end{bmatrix}\right) \\
&= \begin{bmatrix} O_{21} \\ O_{22} \end{bmatrix} \\
&= a^{[2]}
\end{aligned}
$$

Inside Output Layer

Inside Output Layer

$$\sigma\left(\begin{bmatrix} W_{311} \\ W_{321} \end{bmatrix}^T \cdot \begin{bmatrix} O_{21} \\ O_{22} \end{bmatrix} + [b_{31}]\right)$$

$$= \sigma\left([W_{311} \cdot O_{21} + W_{321} \cdot O_{22}] + [b_{31}]\right)$$

$$= \sigma\left(W_{311} \cdot O_{21} + W_{321} \cdot O_{22} + b_{31}\right)$$

$$= Y^i = [O_{31}]$$

$$= a^{[3]}$$

Perceptron learning

Steps in Feedforward Neural Networks Work

- Initialization: Initialize weights ww and biases bb to small random values.
- Forward Propagation: Calculate the output of the network using the current weights and biases., take 1st input of training data
- Loss Calculation: Compute the loss (error) using a loss function such as mean squared error (MSE) for regression or cross-entropy for classification.
- Backpropagation: Calculate the gradient of the loss with respect to each weight using the chain rule.
- Update the weights and biases to reduce the loss using an optimization algorithm like gradient descent.
- Iteration: Repeat forward propagation, loss calculation, and backpropagation for multiple epochs until the network converges or a stopping criterion is met.

So • loss $\Rightarrow \left[\frac{1}{2}(y_i - \hat{y}_i)^2\right]$ for any particular input point.

for any $\Rightarrow \left(w^{new} = w^{old} - \eta \frac{\partial L}{\partial w^{old}}\right)$

$\hat{y} = f(w_7 o^1 + w_8 o^2 + b_3)$

$w_8^{new} = w_8^{old} - \eta \frac{\partial L}{\partial w_8^{old}}$

$\frac{\partial L}{\partial w_8^{old}} \Rightarrow \left(\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_8}\right)$

$\frac{\partial L}{\partial w^8} \Rightarrow \left[-(y_i - \hat{y}_i) f'(w_7 o^1 + w_8 o^2 + b_3) o^2\right]$

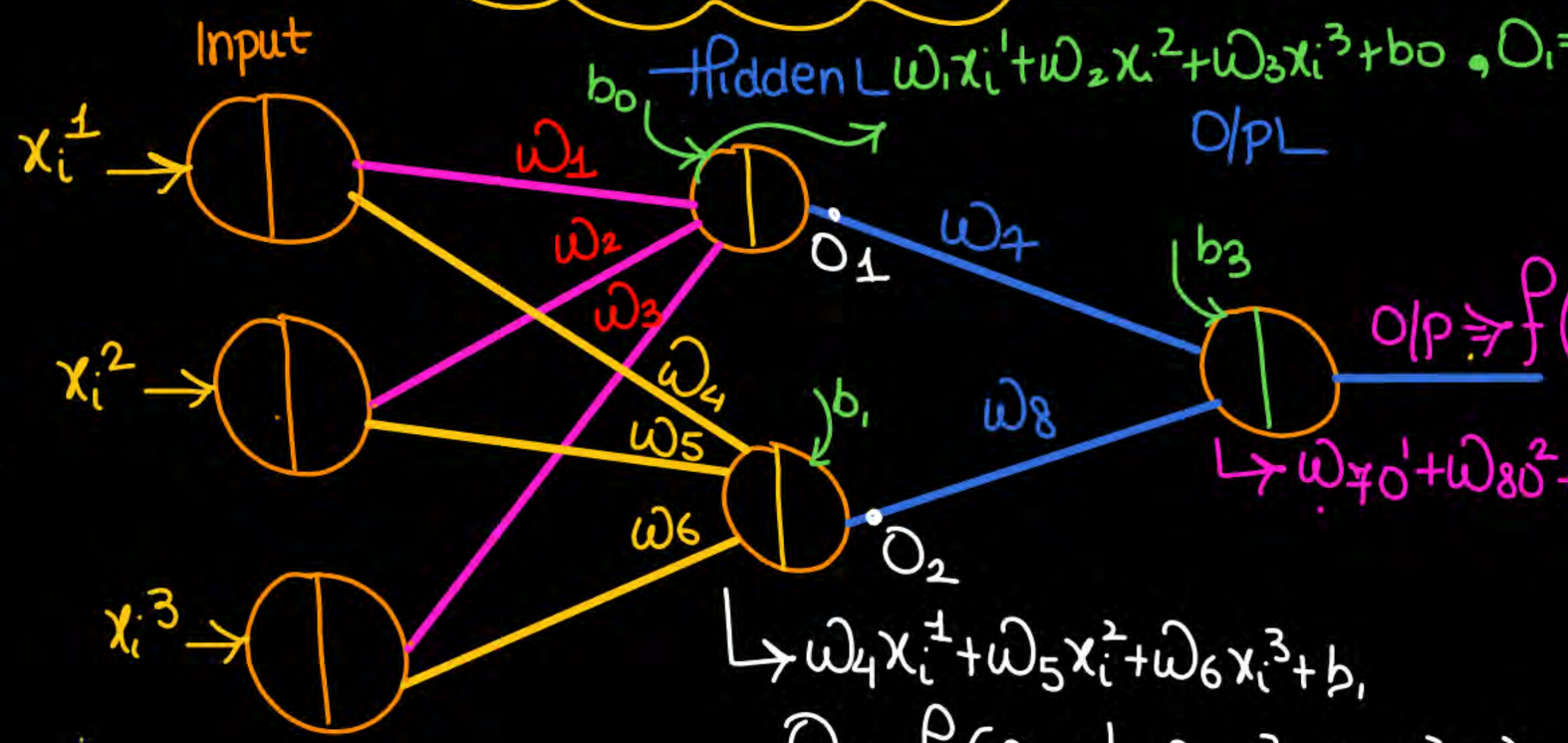
Perceptron learning

• we start with Random weight values.

Feedforward Phase

Ex

So input is 3D input



$w_1x_i^1+w_2x_i^2+w_3x_i^3+b_0$, $O_1=f(w_1x_i^1+w_2x_i^2+w_3x_i^3+b_0)$

$O/P \Rightarrow f(w_7O^1+w_8O^2+b_3)$

$\hookrightarrow w_7O^1+w_8O^2+b_3$

$\hookrightarrow w_4x_i^1+w_5x_i^2+w_6x_i^3+b_1$

$O_2=f(w_4x_i^1+w_5x_i^2+w_6x_i^3+b_1)$

THANK - YOU